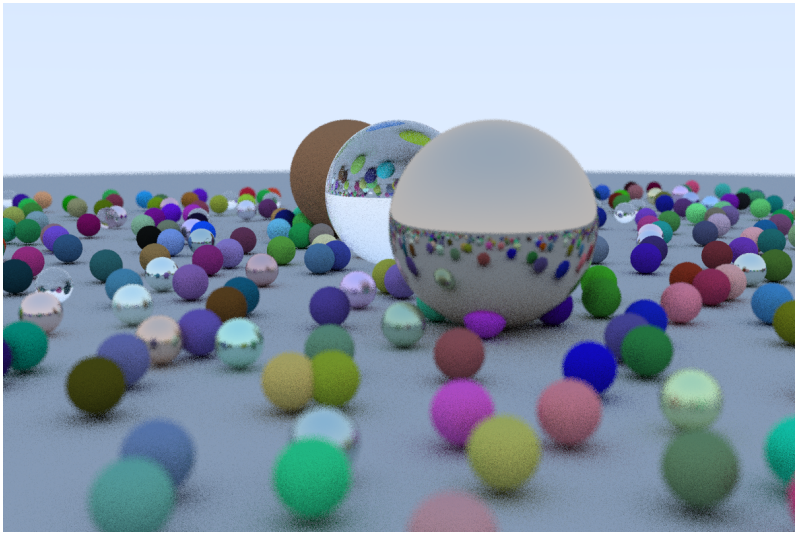
Applied GPU Programming Project : Accelerated Ray Tracing

Antonutti Adrien - adrien.antonutti@student.uclouvain.be
Duke-Bergman Kei - keidb@kth.se
Prete Giovanni - prete@kth.se

January 7, 2026



1 Introduction

The problem addressed in this work is ray tracing, a rendering technique used to generate highly realistic images and one that has seen increasing adoption in recent years. In ray tracing, a ray is cast for each pixel of the image and traced through the scene to determine which object it intersects. The colour of the pixel is then computed based on the properties of the intersected object, such as its material characteristics. For certain materials, including reflective and transparent ones, additional rays/bounds are needed to model effects like reflection and refraction. This behaviour is governed by the material's scattering process, which determines how rays interact with the surface.

1.1 Description of initial implementation

The initial proposed implentation by Roger Allen is quite simple but convenient to learn ray tracing. It is a quite direct CUDA translation of the book *Ray Tracing in One Weekend* implemented in C++.

1.1.1 Initialisation and world creation

The frame buffer is initialised with *unified memory* which is accessible both by the CPU and GPU and manual transfer of data between host and device is managed automatically. *cuRAND* is used for the random ray direction in scattering process and random scene generation. This library allow random number to be generated on the GPU. The kernel *rand_init* initialise the random state. Then, the kernel *create_world* generate the world by instantiate all the scene's object and place them into a *hitable_list*, which is a wrapper containing all the hittable objects in the scene. A camera is also created with all needed properties.

1.1.2 Rendering

Rendering is done with *render* kernel that use blocks size of 8x8 and is done on the 2D pixel grid. For each pixel, *ns* samples are done. Ray tracing is based on a Monte-Carlo approximation, therefore the higher the number of samples the higher the image quality will be. The default value used was 10. Each of the sample used the *color* function to determine the sample's colour. The *color* function can bounce the ray up to 50 times. It can be less

if the material absorb all the light or if the sky is hit. Once the *ns* samples are done, the kernel average and normalize them before writing them in the framebuffer.

1.1.3 Image saving and clean-up

Once the rendering kernel is finished, the buffer is saved in PPM format[1]. Finally, all the allocated objects are freed with the *free_world* kernel. Only one thread is working for this kernel : resources are freed serially.

1.2 Profiling of initial implementation

To identify which part of the program could be sped up, each major part of the program was profiled. The program step were identified as followed : *memory allocation*, *world creation*, *rendering*, *image saving* and *resource freeing*. Profiling was done with multiple number of objects in the size to see the impact of larger amount of object on execution time. Figure 1 shows the obtained results. It shows that the majority of time is spent during the rendering of the image, which is expected. For a small number of objects, the world creation is negligible; however for larger scene, the world creation start to take more time and become not negligible. All the others parts of the program are negligible for this basic implementation.

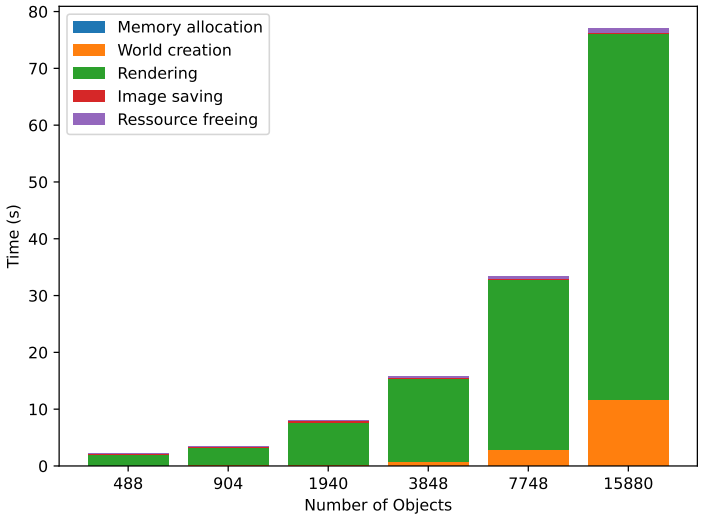


Figure 1: Profiling of initial implementation

The occupancy of the *render* kernel was 42.64%.

2 Methodology

2.1 Bottleneck identification

2.1.1 Hitable list

A potential bottleneck is the *hit* function of the *hitable_list* wrapper for the scene : each time we trace a ray it is needed to iterate trough all the objects of the scene to known which object is hit. This explain the poor performance observed in figure 1 for large amount of objects. This is especially important since this hit function will be called a huge amount of time : the maximum number of times the whole list will be iterated is : $n_x \cdot n_y \cdot n_s \cdot max_{bound}$ with n_x and n_y the number of pixels, n_s the number of samples per pixel and max_{bound} the maximum number of bounds possible for each ray. For our default parameter, this is equal to 480 millions. In reality, a lot of rays will stops earlier than that and this number is over estimating the number of call. However, optimizing this function would lead to better performance.

2.1.2 Parallel world creation and clean-up

By inspecting the code, it can be seen that the kernel *create_world* and *free_world* are launched on the GPU and are running on only one thread. It could be faster by using multiple threads for world initialisation and clean-up. In the current implementation, it is not the main bottleneck but replacing by a parallelized kernel could result in better performance.

[1]In original implementation, PPM was print to stdout directly, but we changed that early to be more convenient and not related to course subject.

2.2 Optimisation

The first (and minor) optimisation was to change the frame buffer type *float* to *u_int8_t*. This reduce the amount of time when fetching the buffer back to the CPU. However it won't improve the execution time since the memory transfer is negligible, as shown in previous section.

2.2.1 BVH implementation

Bounding volume hierarchy (BVH) address the hitable list issue explained in section 2.1.1. It consists of creating a tree where each node correspond to a box that contains all objects inside this box. All child node are contained within the box of the parent node. This acceleration structure reduce the complexity of checking which object is hit from $\Theta(n)$ to $\Theta(\log(n))$. This acceleration is possible with the added cost of computing the tree.

Tree data structure

To implement the BVH, 2 additional structures/class were needed : *aabb* and *BVHNodeData*. Class *aabb* represent a box characterized by only 2 points : *min* and *max*. This class will allow to define the boxes that will contains our object to create the tree. For the nodes of the three, the structure *BVHNodeData* is used. It has the regular attributes of a tree node : left and right child index (all nodes will be stored in an array), the box linked to the node. For leaf nodes, two additional attribute are needed : *start* indicates where the primitives of the node starts in the array and *count* the number of primitives. Each leaf node contain multiple object (4 in our implementation) with the goal to leverage coalesced memory access of GPU. As the node are stored in an array, leaf will be made of 4 consecutive entry of this array. When loading a leaf node, the next node of the leaf might be loaded into GPU cache.

Tree traversal

The tree traversal is a regular tree search. Since the tree is stored in an array a stack and stack pointer are used to keep track of nodes that need to be visited. It can be described as follow for a given node and ray :

- Get next node index on the stack (if available) and recover node data
- Try to hit the box of the node, if ray miss the box, stop and don't explore child node.
- If node is a leaf, look for the nearest object and return it (false if no object)
- Else, put child node on the stack to explore them.

Tree creation

The tree creation is mainly done on the CPU because recursion is easier to implement on CPU. There are possible implementation of BVH tree creation on GPU but those are harder to implement. Before actually computing the BVH tree, the bounding boxes (*aabb*) for each object are computed on the GPU with the kernel *compute_bounding_boxes*. Then the boxes are transferred to the host memory. This avoid to transfer the useless data of objects for the BVH computation : only the bounding box of each object is needed. After that the BVH tree construction is built using the following steps :

- Compute bounds for input objects (objects that will contained within the node's box).
- Create leaf if amount of input object is small enough.
- Try splitting along the 3 axes (x,y and z) and choose the split with higest surface area heuristic (SAH[2])
- Sort object along chosen axis.
- Build the (non-leaf) node.
- Recursively create the right and left child node with the chosen split.

2.2.2 Parallel generation and clean-up of the scene

The last optimisation was to parallelise the world creation and clean up of the scene. The 2 nested loops of the world creation have been replaced by a kernel working on a 2D grid. With this improvement, it was not possible to generate the exact same image (ie the sphere at same position and colour). Therefore the reference implementation with only the parallel generation is available in *ref_src/main_parallel_gen.cu*. This version will be used in the next section for the validation of our optimised implementation. For the world clean up, objects are freed in parallel using a 1D kernel, replacing the simple loop that was running on one thread.

2.2.3 Static Dispatch and Manual Material Management

A critical performance bottleneck in the reference implementation was the use of C++ virtual functions for both geometry intersection (`hitable`) and material scattering (`material`). On GPU architectures, dynamic dispatch via virtual function tables (vtables) introduces significant latency and instruction divergence, as the hardware cannot easily predict the target of a virtual call within a warp.

To mitigate this, we transitioned from a polymorphic class hierarchy to a **Static Dispatch** model based on **Type Tagging**. Specifically:

[2]SAH is an heuristic and is trying to estimate the best split.

- **Tagged Union for Materials:** The abstract `material` class was replaced by a flat `material_opt` structure. This structure uses a `MaterialType` enumeration (acting as a type tag) to identify the scattering logic. The previously virtual `scatter` function was refactored into a single function using a `switch` statement. This allows the compiler to optimize the branch logic and potentially leverage jump tables or predicated execution, significantly reducing the overhead of material evaluation.
- **Type-Specific Intersection:** The `hitable` interface was flattened to allow direct calls to geometry-specific intersection routines. In the optimized BVH traversal, the engine interacts directly with `sphere_opt` objects rather than through a pointer to a base class. This removal of the virtual layer ensures that the `hit` function can be inlined by the compiler, improving register allocation and reducing memory pressure during the traversal of the acceleration structure.

2.3 Impact of unified memory

For the naive unified memory evaluation, performed before integrating with BVH, three implementations were considered: One using cudaMalloc for all data structures formed on the GPU, the standard implementation where the framebuffer resides in unified memory, and an implementation where all memory allocations were directed to unified memory.

Each implementation was tested 4 times, over image sizes 512*512, 1024*1024, 2048*2048, 4096*4096. The time taken for each major operation within the code was tracked. The tracked items were: 1. Frame buffer allocation (memory allocation), 2. Allocation of random state (memory allocation) 3. Initialization of random state (kernel) 4. Initialization of world state (kernel) and 5. the Image Creation Kernel (kernel). Each test consisted of repeatedly running the kernel 5 times and calculating the average of the tracked items. Prefetching and Memadvise were attempted, but the operating system on the experimental setup (Windows) does not support these options, and so they had to be abandoned. Validation of results using Mean Square Error and Structural Similarity Index Measure was not performed, since unified memory should not affect the image contents itself. The experiment was repeated once BVH had been implemented. This time, the resolution was set to 1200x800, and the number of objects was varied, to stay coherent with the profiling approaches done since starting the work on BVH. The tested object amounts were 904, 1940, 3848 and 7748. The measured steps were 1. Memory Allocation, 2. Scene generation, 3. BVH generation, 4. Render initialization, 5. Rendering and (when applicable) 6. Memcpy.

3 Experimental setup

In the unified memory evaluation, test were performed on a custom built PC with an AMD Ryzen 7 5800X processor, 16GB of ram at 3200MT/s, and an NVIDIA GeForce RTX 3080 with 10GB of onboard VRAM. Programs were run through nvcc on Ubuntu WSL.

For reference implementation profiling and optimised implementation, the program ran on a custom server PC with following specs : Intel(R) Xeon(R) W-2123 CPU, 96GB of ram at 2666Mhz and a NVIDIA GeForce GTX 1080 Ti with 12GB of VRAM. The CUDA version used was 12.2 and NVIDIA driver version 535.

4 Results

4.1 Optimisation implementation result

4.1.1 Validation of optimised implementation

To ensure that our optimised implementation is correct, two metrics were used : mean square error (MSE) and structural similarity index measure (SSIM). MSE was implemented as folowed :

$$MSE = \frac{1}{3N}\sum_{i=1}^{N}\sum_{c=1}^{3}\left(\frac{x_{ic} - y_{ic}}{255}\right)^2$$

where N is the number of pixel, x_{ic} denotes the cth color channel value of ith pixel of image x between 0 and 255. This MSE is normalised between 0 and 1. Peak signal to noise ratio (PSNR) is derived from MSE :

$$PSNR = 10\log_{10}(MSE)$$

. MSE is a good indicator for image similarity, but it may not reflect human perception. Structural similarity index measure aims to represent better what the human eye perceive. It is computed as followed :

$$SSIM = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x + \sigma_y + C_2)}$$

μ_y and μ_y are the mean value of the image, for a certain window. In our case the windows was 8x8 pixels. σ_x and σ_y is the variance of the image within the window and σ_{xy} the covariance. C_1 and C_2 are constant to avoid division by zero. The SSIM is computed for all window possible on the image and average value is computed. A value close to 1 indicates that two image are really close, and close to 0 really different.

n_s	**MSE (%)**	**PSNR (dB)**	**SSIM (%)**
10	0.11	29.5	99.54
20	0.0736	31.33	99.69
50	0.0475	33.23	99.80

Table 1: Similarity metrics comparaison for different number of sample per pixel n_s

Table 1 shows that the generated images are really similar : MSE is less than 1% and SSIM is really close to 100%. They are a little bit different, this is a bit strange : BHV only change the method of checking which object the ray hit, it shouldn't change the output. Our assumption is that the order of float operation is not the same with both implementation and that it leads to slightly different image. An interesting trend is that the more sample per pixel is used, the closer both image are. This comfort our idea of float precision difference between both implementation. When more samples are done, the little difference caused by float precision are removed by the high number of samples. This also indicate that our BVH implementation is correct. Visually, the difference between 10 and 20 samples is noticeable. However, the difference between 20 and 50 samples is really subtle.

4.1.2 Profiling of optimised implementation

As shown in figure 2, massive speed-up is achieve with the BVH implementation[3]. For each number of object, 5 runs were done and only the average is displayed for each implementation : reference, BHV only and BVH and parallel scene generation. For the default (and smallest) scene with 488 objects, the execution is around 7 times faster. For this scene size, the parallel scene generation is not a bottleneck already, so the speed up below $\approx$ 1000 objects is negligible. For larger scene, the benefit of the BVH is even more important : for a scene of 8000 objects, the rendering time goes from $\approx$ 32 to 3.12 seconds, and with parallel scene generation, the time is reduced to $\approx$ 0.36 seconds. This is a 88.88 speed up. For even larger scene, the speed up is larger, reaching around 200. This is explained by the BVH reduced complexity ($\Theta(n)$ vs $\Theta(\log n)$. This graph also show the benefit of parallel scene generation for large scene, as the speed-up for the BVH + parallel scene generation versus BHV only is 31 (for the largest scene).

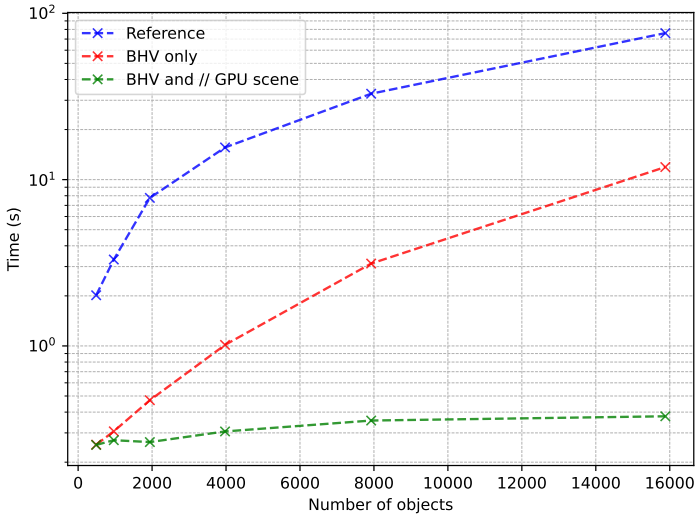


Figure 2: Execution time comparison of reference vs BVH vs BVH and parallel world generation

Figure 2 is not showing the parallel clean up of resource benefit. It is however noticeable when profiling with *nvprof*. For a scene with 7748 objects, the time required for the *free_world* kernel is 331.10 ms for the reference implementation and 1.3797 ms for our optimisation. For other number of object, the result is the same, parallel resource clean-up is much faster. This is a small gain compared to the one of BVH and parallel world generation but it is still worth mentioning.

Figure 3 shows the profiling of major step of the program for the optimised version. It can be seen that the time to render an image is much smaller than before (see figure 1). The rendering time is a bit larger for large

[3]Note that those results doesn't include image saving and resource clean up, as they are minimal

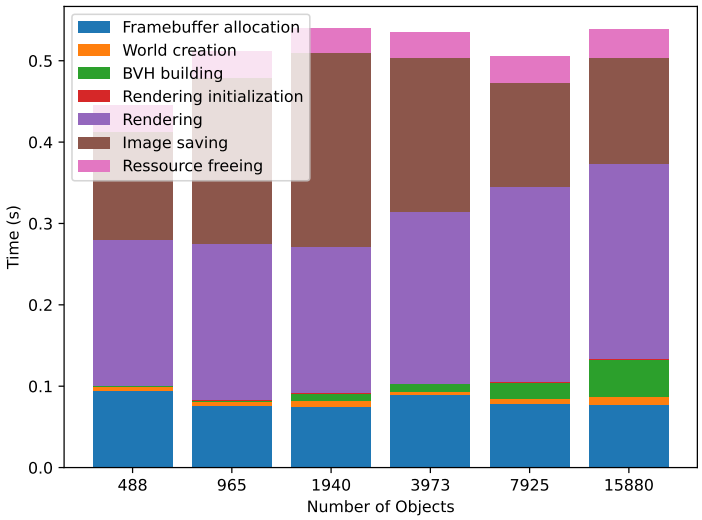


Figure 3: Profiling of optimised implementation

amount of object, but the difference is quite small : *render* kernel take 0.18 seconds with 488 objects and 0.238 seconds with 15880 objects. *render* kernel is much faster, but there is some overhead to that : the time needed to construct the BVH. For small amount of object, this time is negligible. For larger scene, the time needed for the tree construction is higher and is not negligible. However, this small overhead is largely beneficial on the global execution time, as shown previously. The time for the world creation is now negligible, even for large amount of objects, which was not the case in first implementation. Thanks to the massive speed up of rendering and world creation, parts that were negligible before (frame buffer allocation, image saving and resource clean up) aren't any more. Frame buffer allocation and image saving take more times than rendering initialisation, world creation and BVH building together. For the image saving, the time needed should be constant as the image size are constant. This is not what has been measured and is probably due to the scheduling of the OS.

4.2 Static dispatch Performance Evaluation

The removal of virtual functions, combined with the BVH optimization, resulted in a drastic performance improvement. The following analysis breaks down the results across different scenarios.

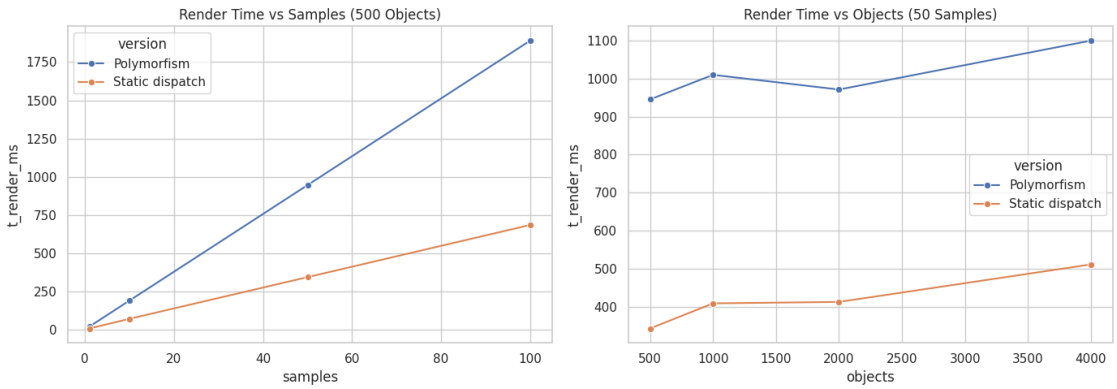


Figure 4: Comparison between Polymorfism and Static dispatch render time across different number of sample.

Figure 4 illustrates the linear scaling of both implementations with respect to the number of samples. However, *Giovanni_Opt* maintains a much lower growth rate, confirming its efficiency in high-workload scenarios.

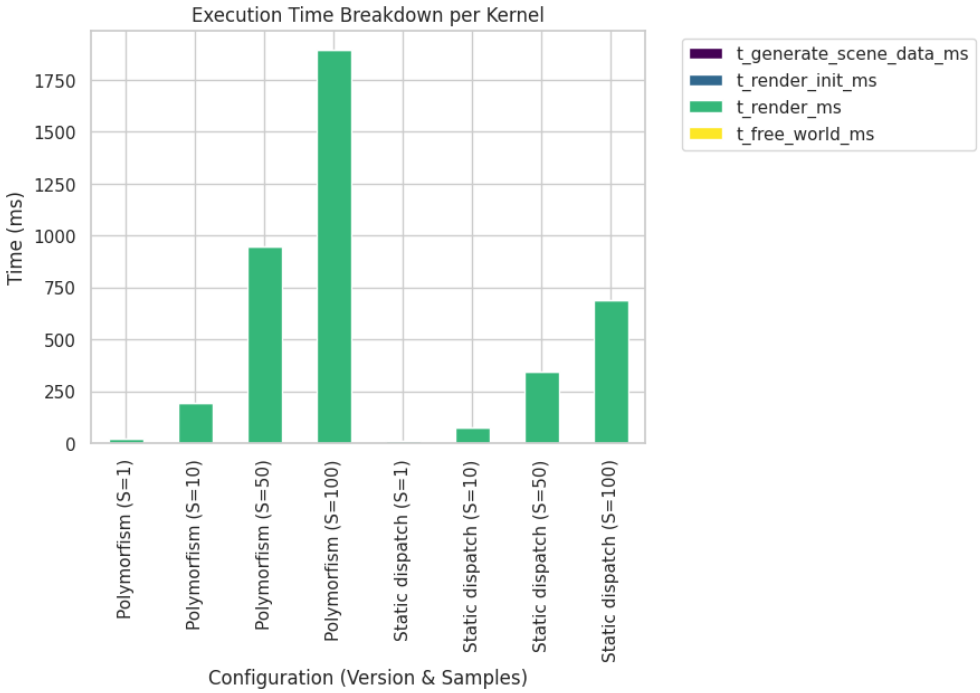


Figure 5: Kernel Time Breakdown: Distribution of time across the different pipeline stages.

Finally, Figure 5 provides a breakdown of the total execution time. While the `render` kernel remains the dominant part of the program, the absolute time spent is significantly lower in the optimized version, effectively shifting the bottleneck towards initialization and memory allocation for smaller sample sizes. Note that with this optimisation, the MSE and SSIM metrics results are exactly the same, as expected since the logic is exactly the same.

4.3 Unified memory vs standard memory allocation

The explicit memory transfer approach is faster than the two other naive unified memory approaches. These discreptancies are displayed in 6. On windows operating systems, smaller pieces of managed data cannot be moved to the GPU on-demand. Instead all pieces of managed memory have to reside in the same space. GPU memory oversubscription is also not supported, neither is simultaneous access to managed memory, which eliminates multiple performance gains. Without proper management, the page faults from unified memory are slower than explicit mermory movement.

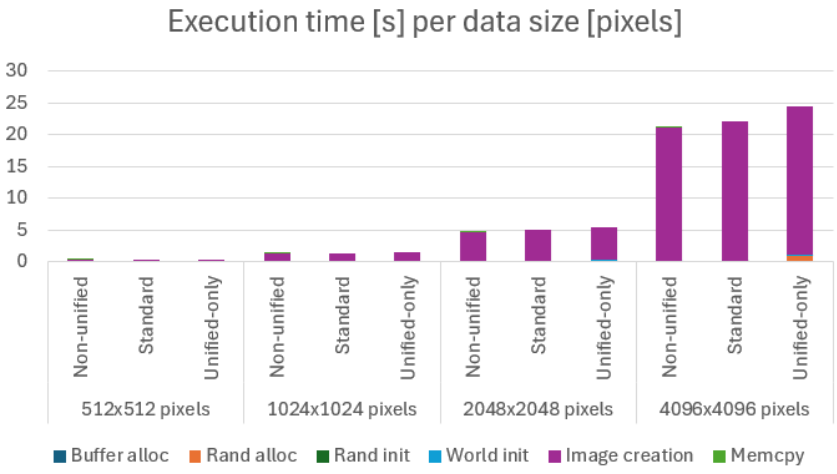


Figure 6: Run times of unified memory approaches

However, when implemented on top of the BVH system and evaluated for amount of objects, as shown in 7, the reference approach of using unified memory for the frame buffer and explicit memory allocation on the GPU instead performs better than a purely explicit declaration or purely unified memory approach, implying that this approach may be better in this implementation.

Figure 7: Run times of unified memory approaches

5 Conclusions

In this project, we explored several GPU-focused optimizations to improve the performance of a ray tracing implementation. By identifying rendering as the main bottleneck, we showed that using a BVH acceleration structure significantly reduces execution time, especially for large scenes. Additional improvements, such as parallel scene generation and the removal of virtual function calls through static dispatch, further increased performance while preserving image quality. The study also highlighted how different memory management strategies can impact runtime. Overall, the results confirm that adapting both algorithms and code structure to GPU architectures is essential for efficient ray tracing. Further improvements could be experimented, such as BVH tree creation directly on GPU.

Contributions

Adrien performed the float to u_int8_t change and implemented BVH, parallel generation and clean-up of the scene. Kei implemented and evaluated three levels of unified memory, and attempted memadvise and prefetch in Windows. Giovanni implemented the Static dispatch version, focusing on the removal of "Virtual Hell" by replacing polymorphic classes with enum-based structures and direct type calls.

References

[1] *Accelerated Ray Tracing in One Weekend in CUDA*. NVIDIA Technical Blog. 5 **november** 2018. URL: `https://developer.nvidia.com/blog/accelerated-ray-tracing-cuda/` (**urlseen** 28/12/2025).

[2] Peter Shirley. "Ray Tracing in One Weekend". **in**().

[3] Peter Shirley. "Ray Tracing: The Next Week". **in**().

[4] *Unified Memory for CUDA Beginners*. NVIDIA Technical Blog. 20 **june** 2017. URL: `https://developer.nvidia.com/blog/unified-memory-cuda-beginners/` (**urlseen** 28/12/2025).